

Polynomial Interpolation: Newton & Lagrange Methods and the Runge Phenomenon

A small comparative study with some plots

Author:

Otmane Belkhair

Master Data & IA

Institut National des Postes et Télécommunications (INPT)

October 2026

1 Introduction

The idea of this report is to look at two classic ways to do polynomial interpolation: Lagrange and Newton. We take a function, pick some points on it, and try to rebuild a polynomial that goes through those points.

The function we use is the famous Runge function:

$$f(x) = \frac{1}{1 + 25x^2}, \quad x \in [-1, 1]. \quad (1)$$

With only a few points everything looks fine, but when we add more points (still equally spaced) we start seeing big oscillations near the edges. This is called Runge's phenomenon, and the goal is just to show it clearly with the two methods.

2 Theoretical Background

2.1 Lagrange Interpolation

In the Lagrange method we write the polynomial as a sum of the y values multiplied by some basis polynomials:

$$P_n(x) = \sum_{i=0}^n y_i \ell_i(x), \quad (2)$$

where

$$\ell_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{x - x_j}{x_i - x_j}. \quad (3)$$

It is simple to understand but a bit slow if we evaluate it many times, and adding a new point means recomputing almost everything.

2.2 Newton Interpolation

Newton uses divided differences. The polynomial looks like this:

$$P_n(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \cdots + a_n \prod_{j=0}^{n-1} (x - x_j), \quad (4)$$

with

$$f[x_i, x_{i+1}, \dots, x_{i+k}] = \frac{f[x_{i+1}, \dots, x_{i+k}] - f[x_i, \dots, x_{i+k-1}]}{x_{i+k} - x_i}. \quad (5)$$

Once we have the table, evaluating the polynomial is cheap, and it is easier to add new points later.

2.3 Runge's Phenomenon

If the points are equally spaced on $[-1, 1]$, high-degree polynomials can oscillate a lot near the ends even if the original function is smooth. That is the Runge phenomenon. Using Chebyshev points or splines usually fixes it.

3 Implementation

Here is the code we used (pure Python).

3.1 Lagrange

```

1 def lagrange_interpolation(x_values, y_values, x_target):
2     polynome = 0
3     n = len(x_values)
4     for i in range(n):
5         Li = 1
6         for j in range(n):
7             if i != j:
8                 Li *= (x_target - x_values[j]) / (x_values[i] - x_values[j])
9         polynome += y_values[i] * Li
10    return polynome

```

3.2 Newton

```

1 def newton_interpolation(x, y, value):
2     n = len(x)
3     table = [[0.0 for _ in range(n)] for _ in range(n)]
4     for i in range(n):
5         table[i][0] = y[i]
6     for j in range(1, n):
7         for i in range(n - j):
8             table[i][j] = (table[i+1][j-1] - table[i][j-1]) / (x[i+j] - x[i])
9     result = table[0][0]
10    product = 1
11    for j in range(1, n):
12        product *= (value - x[j-1])
13        result += table[0][j] * product
14    return result, table

```

4 Numerical Experiments

We always use the Runge function on $[-1, 1]$ and a fine grid of 500 points to draw the curves.

4.1 Case with 5 points

With only 5 equally spaced points the approximation is still decent. Both methods give exactly the same polynomial (as they should) and the curve follows the real function quite well in the middle.

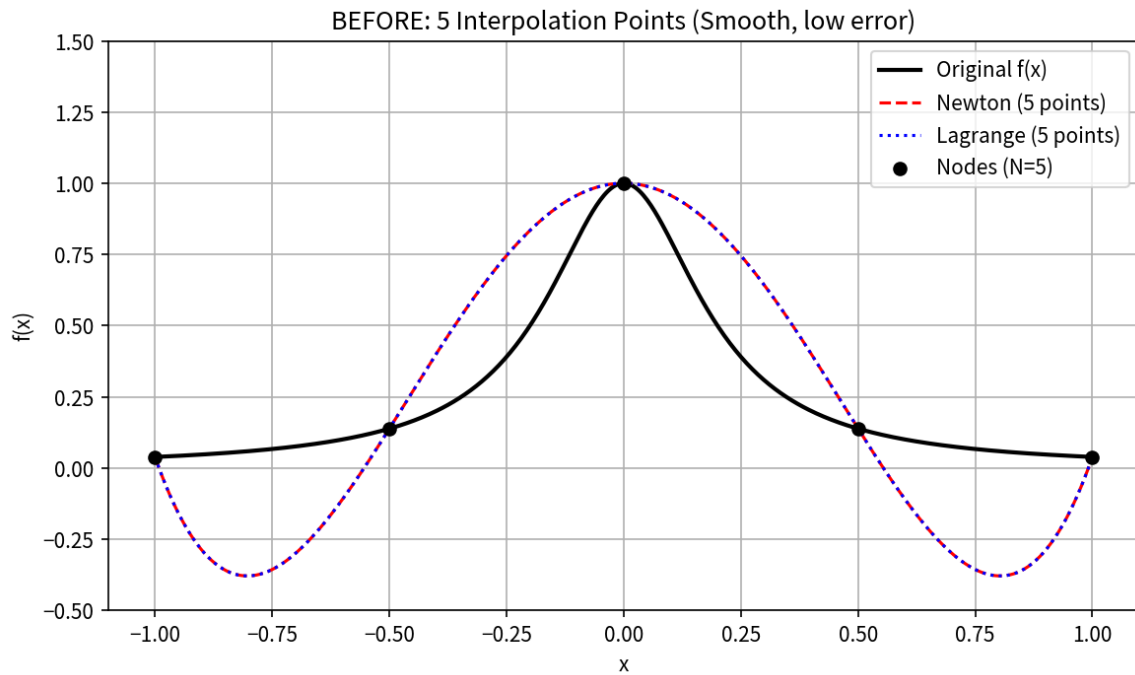


Figure 1: 5 interpolation points. Newton and Lagrange give the same result and stay close to $f(x)$.

4.2 Case with 11 points

When we go to 11 points the oscillations become obvious near $x = \pm 1$. The polynomial still passes through all the nodes, but between them (especially at the ends) it goes wild. This is the classic Runge phenomenon.

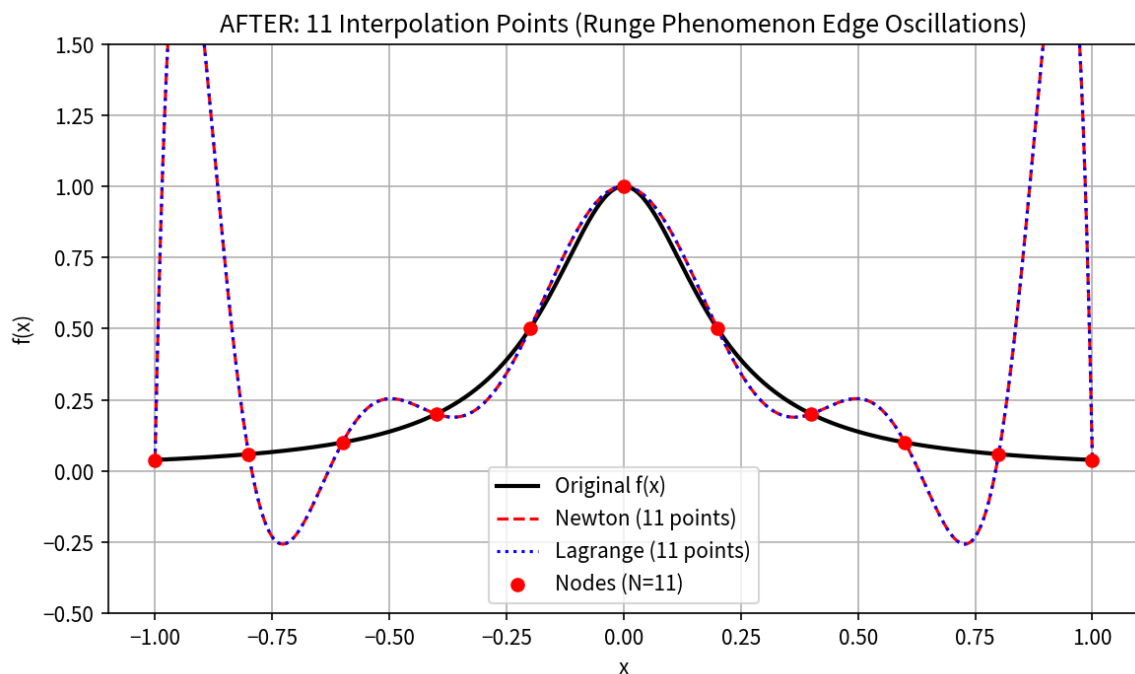


Figure 2: 11 interpolation points. Clear oscillations at the borders (Runge phenomenon).

5 Conclusions

Both Lagrange and Newton build the same unique polynomial, so the plots look identical.

With few points everything is fine, but with more equally spaced points we get big oscillations near the edges. That is Runge's phenomenon.

In practice it is better to avoid high-degree polynomials on equispaced nodes. Using Chebyshev points or just switching to splines is usually a safer choice.

Report made by Otmane Belkhair – Master Data & IA, INPT